INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Práctica 4: Text Classification

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

FECHA DE ENTREGA: 30 DE ABRIL DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 18 DE ABRIL DE 2026

Marco Teórico

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) es una rama de la inteligencia artificial que se enfoca en el desarrollo de métodos computacionales capaces de analizar, interpretar y generar lenguaje humano de forma automática. Esta disciplina integra conocimientos de lingüística, aprendizaje automático, estadística y ciencias de la computación para permitir que las computadoras procesen grandes volúmenes de información textual. Entre sus aplicaciones más relevantes se encuentran la clasificación de documentos, el análisis de sentimientos, la traducción automática y los sistemas de recomendación.

Una de las tareas fundamentales dentro del NLP es la clasificación de texto, la cual consiste en asignar una etiqueta a un documento en función de su contenido. Este problema se aborda comúnmente mediante técnicas de aprendizaje supervisado, donde un modelo es entrenado a partir de ejemplos previamente etiquetados para aprender patrones que le permitan clasificar nuevos textos. En el caso de la clasificación binaria, el modelo debe decidir entre dos posibles clases.

En esta práctica se trabaja con el dataset amazon_polarity, el cual contiene reseñas de productos de Amazon clasificadas como positivas o negativas. Este tipo de problema también puede interpretarse como una tarea de análisis de sentimiento, ya que el objetivo es identificar la polaridad de la opinión expresada en el texto. El dataset utilizado es de gran escala, con millones de ejemplos, lo que implica la necesidad de utilizar métodos eficientes tanto en representación como en entrenamiento.

Para que los algoritmos de aprendizaje automático puedan procesar texto, es necesario transformarlo en una representación numérica. Una de las técnicas más utilizadas es TF-IDF (Term Frequency - Inverse Document Frequency), la cual calcula la importancia de cada término dentro de un documento en relación con un corpus completo. Esta técnica combina la frecuencia de aparición de una palabra dentro de un documento con su rareza en el conjunto de documentos, permitiendo destacar términos relevantes y reducir el impacto de palabras comunes.

Además, es posible enriquecer la representación del texto mediante el uso de n-gramas, que permiten capturar secuencias de palabras consecutivas. En particular, el uso de unigramas y bigramas permite incorporar información de contexto local, lo cual es especialmente útil en tareas de análisis de sentimiento, donde combinaciones como "not good" o "very bad" tienen un significado distinto al de las palabras individuales.

El proceso de representación del texto suele ir acompañado de una etapa de preprocesamiento, cuyo objetivo es limpiar y normalizar los datos. Entre las operaciones más comunes se encuentran la conversión a minúsculas, la eliminación de caracteres especiales y la eliminación de palabras vacías (stopwords). La eliminación de stopwords

reduce el ruido en los datos al eliminar palabras frecuentes que no aportan información relevante para la clasificación, mejorando así la eficiencia y el desempeño del modelo.

En cuanto a los algoritmos de clasificación, en esta práctica se emplearon dos modelos principales. El primero es SGDClassifier (Stochastic Gradient Descent Classifier), un modelo lineal que optimiza su función de pérdida mediante descenso por gradiente estocástico. Este algoritmo es especialmente adecuado para problemas de gran escala y espacios de alta dimensionalidad, como ocurre en la clasificación de texto, ya que permite entrenamientos rápidos y eficientes incluso con millones de ejemplos.

El segundo modelo utilizado es Complement Naive Bayes, una variante del clasificador Naive Bayes diseñada para mejorar el rendimiento en tareas de clasificación de texto. Este modelo se basa en un enfoque probabilístico y es ampliamente utilizado como referencia en problemas de minería de texto debido a su simplicidad y rapidez.

Para evaluar el desempeño de los modelos se utilizan diversas métricas. La accuracy mide la proporción de predicciones correctas, mientras que la precision indica qué proporción de los ejemplos clasificados como positivos son realmente positivos. Por su parte, el recall mide la capacidad del modelo para identificar correctamente los ejemplos positivos. Una métrica ampliamente utilizada es el F1-score, que combina precisión y recall en una sola medida:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Además de estas métricas, también se emplean medidas como ROC-AUC y PR-AUC, que permiten evaluar el comportamiento del modelo en distintos umbrales de decisión, así como la matriz de confusión, que muestra de manera detallada los aciertos y errores del clasificador.

En conjunto, estas técnicas permiten construir un sistema capaz de analizar automáticamente grandes volúmenes de texto y clasificar reseñas de acuerdo con su polaridad. Este tipo de aplicaciones es fundamental en contextos reales, donde es necesario procesar grandes cantidades de opiniones de usuarios para apoyar la toma de decisiones en plataformas digitales.

Material y Equipo

Para la realización de esta práctica se utilizaron diversas herramientas de software orientadas al procesamiento de lenguaje natural y al aprendizaje automático. En primer lugar, se empleó el lenguaje de programación Python, el cual es ampliamente utilizado en aplicaciones de ciencia de datos e inteligencia artificial debido a su facilidad de uso y a la gran cantidad de bibliotecas especializadas disponibles.

El desarrollo del sistema de clasificación se llevó a cabo utilizando varias bibliotecas de Python. Una de las principales fue scikit-learn, la cual proporciona herramientas para la implementación de algoritmos de aprendizaje automático, incluyendo modelos de clasificación, métodos de evaluación y técnicas de representación de texto como TF-IDF. También se utilizó la biblioteca datasets, que permitió cargar de manera eficiente el conjunto de datos amazon_polarity, facilitando el manejo de grandes volúmenes de información textual.

Adicionalmente, se empleó la biblioteca pandas para la manipulación y organización de datos en estructuras tipo tabla, lo que permitió generar y almacenar los resultados obtenidos, como métricas de evaluación, matrices de confusión y tablas comparativas entre modelos. La biblioteca numpy fue utilizada para realizar operaciones numéricas eficientes sobre los datos, especialmente durante el manejo de etiquetas y cálculos internos del modelo.

Para la visualización de resultados se utilizó matplotlib, que permitió generar gráficas como las curvas ROC y Precision-Recall, facilitando la interpretación del desempeño de los modelos. Asimismo, se utilizó el módulo re de Python para realizar operaciones de limpieza del texto mediante expresiones regulares, eliminando caracteres innecesarios y normalizando el contenido textual.

El conjunto de datos utilizado en esta práctica fue amazon_polarity, el cual contiene reseñas de productos de Amazon clasificadas en dos categorías: positivas y negativas. Este dataset es de gran escala, ya que cuenta con aproximadamente 3,600,000 ejemplos de entrenamiento y 400,000 ejemplos de prueba, lo que permitió evaluar el desempeño de los modelos en un escenario realista con grandes volúmenes de información.

En cuanto al hardware, la práctica fue realizada en una computadora personal, utilizando un entorno de desarrollo para ejecutar los scripts en Python. Debido al tamaño del dataset, fue necesario considerar el uso eficiente de memoria y tiempo de procesamiento, lo cual influyó en la selección de técnicas de representación y modelos de clasificación adecuados para grandes conjuntos de datos.

Desarrollo de la Practica

En esta práctica se desarrolló un sistema de clasificación de texto utilizando el lenguaje de programación Python, con el objetivo de determinar la polaridad de reseñas de productos del dataset amazon_polarity. El proceso completo incluyó la carga del dataset, preprocesamiento del texto, representación mediante TF-IDF, implementación de modelos de clasificación, entrenamiento y evaluación de resultados.

A continuación, se describe detalladamente cada una de las etapas del desarrollo.

1. Importación de librerías

En primer lugar, se importaron las bibliotecas necesarias para llevar a cabo el procesamiento de texto, la manipulación de datos, la implementación de modelos de aprendizaje automático y la evaluación de resultados.

Estas librerías permiten trabajar con grandes volúmenes de datos, transformar texto en representaciones numéricas y aplicar algoritmos de clasificación de manera eficiente.

```
import os
import re
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from datasets import load_dataset
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import ComplementNB
from sklearn.linear_model import SGDClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    balanced_accuracy_score,
    matthews_corrcoef,
    roc_auc_score,
    average_precision_score,
    log_loss,
    confusion_matrix,
    classification_report,
    roc_curve,
    precision_recall_curve
)
```

Figura 1: Librerías a utilizar

2. Carga del dataset

Posteriormente, se cargó el dataset amazon_polarity utilizando la biblioteca datasets. Este conjunto de datos contiene reseñas de productos clasificadas en dos categorías: positivas y negativas.

El dataset está dividido en dos subconjuntos:

- Conjunto de entrenamiento (train)
- Conjunto de prueba (test)

Además, se trata de un dataset de gran escala, con millones de ejemplos, lo cual permite evaluar el modelo en condiciones realistas.

También se verificó que las clases están balanceadas, es decir, existe la misma cantidad de ejemplos positivos y negativos, lo cual evita sesgos en el entrenamiento del modelo.

```
print("Cargando amazon_polarity completo...")
dataset = load_dataset("amazon_polarity")

X_train_raw = dataset["train"]["content"]
y_train = np.array(dataset["train"]["label"], dtype=int)

X_test_raw = dataset["test"]["content"]
y_test = np.array(dataset["test"]["label"], dtype=int)

print(f"Tamaño train: {len(X_train_raw):,}")
print(f"Tamaño test : {len(X_test_raw):,}")
```

Figura 2: Carga del Dataset

3. Preprocesamiento del texto

Antes de aplicar cualquier técnica de aprendizaje automático, es necesario limpiar y normalizar el texto. Para ello, se implementó una función de preprocesamiento que realiza varias transformaciones sobre los datos.

Entre las principales operaciones realizadas se encuentran:

- Conversión de todo el texto a minúsculas
- Eliminación de saltos de línea
- Eliminación de caracteres especiales mediante expresiones regulares
- Eliminación de espacios redundantes

Este proceso permite reducir la variabilidad del lenguaje y eliminar ruido, facilitando la extracción de patrones relevantes por parte del modelo.

```python
def clean_text(text: str) -> str:
    if not isinstance(text, str):
        text = str(text)

    text = text.lower()
    text = text.replace("\n", " ").replace("\r", " ")
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"[^a-z0-9.,!?;:'\"()\-\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

Figura 3: Función de limpieza de texto

Posteriormente, esta función se aplicó a todos los textos del dataset, tanto de entrenamiento como de prueba..

```python
X_train = preprocess_texts(X_train_raw, USE_TEXT_CLEANING)
X_test = preprocess_texts(X_test_raw, USE_TEXT_CLEANING)
```

Figura 4: Aplicación del preprocesamiento completo

4. Representación del texto con TF-IDF

Una vez preprocesados los datos, se procedió a transformar el texto en una representación numérica utilizando la técnica **TF-IDF**. Esta técnica permite medir la importancia de cada palabra dentro de un documento en relación con todo el conjunto de datos.

Para mejorar la calidad de la representación, se utilizó una configuración optimizada que incluye:

- Uso de unigramas y bigramas, lo que permite capturar relaciones entre palabras consecutivas
- Eliminación de stopwords, para reducir palabras irrelevantes
- Limitación del número de características, con el fin de controlar el consumo de memoria

El uso de bigramas resulta especialmente útil en tareas de análisis de sentimiento, ya que permite interpretar correctamente expresiones como "not good" o "very bad", que cambian completamente el significado del texto.

```
TFIDF_CONFIG = {
    "max_features": 20000,
    "ngram_range": (1, 2),
    "min_df": 5,
    "max_df": 0.90,
    "sublinear_tf": True,
    "strip_accents": "unicode",
    "lowercase": True,
    "stop_words": "english"
}
```

Figura 5: Configuración del vectorizador

5. Definición de modelos de clasificación

En esta práctica se implementaron dos modelos de clasificación con el objetivo de comparar su desempeño:

SGDClassifier

Este modelo está basado en descenso por gradiente estocástico y es especialmente adecuado para problemas de gran escala, ya que permite entrenamientos rápidos incluso con millones de ejemplos.

Se configuró con función de pérdida logística, lo que permite obtener probabilidades y utilizar métricas avanzadas como ROC-AUC.

Complement Naive Bayes

Este modelo es una variante del Naive Bayes, ampliamente utilizado en tareas de clasificación de texto debido a su rapidez y simplicidad. Se empleó como modelo base para comparar su desempeño frente a un modelo más avanzado.

Ambos modelos fueron implementados utilizando pipelines, lo que permite integrar el proceso de vectorización y clasificación en una sola estructura.

```
models = {
    "SGDClassifier_LogLoss_Mejorado": Pipeline([
        ("tfidf", TfidfVectorizer(**TFIDF_CONFIG)),
        ("clf", SGDClassifier(
            loss="log_loss",
            penalty="l2",
            alpha=1e-6,
            max_iter=50,
            tol=1e-3,
            early_stopping=True,
            validation_fraction=0.1,
            n_iter_no_change=3,
            random_state=RANDOM_STATE
        ))
    ]),

    "ComplementNB_Mejorado": Pipeline([
        ("tfidf", TfidfVectorizer(**TFIDF_CONFIG)),
        ("clf", ComplementNB(alpha=0.3))
    ])
}
```

Figura 6: Configuración de los Modelos

6. Entrenamiento de los modelos

Una vez definidos los modelos, se procedió a entrenarlos utilizando el conjunto completo de datos de entrenamiento.

Debido al gran tamaño del dataset, se seleccionaron modelos eficientes capaces de manejar grandes volúmenes de información sin requerir recursos computacionales excesivos.

Durante esta etapa, el modelo aprende a identificar patrones en el texto que le permiten distinguir entre reseñas positivas y negativas.

```
for model_name, model in models.items():
    print(f"\nEntrenando: {model_name}")
    train_start = time.time()

    model.fit(X_train, y_train)

    train_end = time.time()
    print(f"Tiempo de entrenamiento: {(train_end - train_start)/60:.2f} minutos")
```

Figura 7: Entrenamiento de los modelos

7. Evaluación del modelo

Después del entrenamiento, se evaluó el desempeño de los modelos utilizando el conjunto de prueba.

Para ello, se realizaron predicciones y se calcularon diversas métricas que permiten medir la calidad del modelo desde diferentes perspectivas.

Entre las métricas utilizadas se encuentran:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- PR-AUC
- MCC

Estas métricas permiten evaluar tanto la precisión del modelo como su capacidad para identificar correctamente ambas clases.

```
print(f"Evaluando: {model_name}")
eval_start = time.time()

y_pred, y_score, y_proba = get_prediction_outputs(model, X_test)

eval_end = time.time()
print(f"Tiempo de evaluación: {(eval_end - eval_start)/60:.2f} minutos")

metrics = compute_metrics(y_test, y_pred, y_score, y_proba)
metrics["Modelo"] = model_name
metrics["Train Time (min)"] = round((train_end - train_start) / 60, 2)
metrics["Eval Time (min)"] = round((eval_end - eval_start) / 60, 2)

all_results.append(metrics)

cm = confusion_matrix(y_test, y_pred)
cm_df = save_confusion_matrix(cm, model_name)

report_df = classification_report_to_df(y_test, y_pred)
save_dataframe(report_df, f"classification_report_{model_name}.csv")

plot_roc_curve(y_test, y_score, model_name)
plot_pr_curve(y_test, y_score, model_name)

all_predictions[model_name] = {
    "model": model,
    "y_pred": y_pred,
    "y_score": y_score,
    "y_proba": y_proba,
    "confusion_matrix": cm,
    "confusion_matrix_df": cm_df,
    "classification_report_df": report_df
}
```

Figura 8. Predicción de después del entrenamiento

```python
print("\nMétricas:")
for key, value in metrics.items():
    if key != "Modelo":
        if isinstance(value, float) and not np.isnan(value):
            print(f"{key:20}: {value:.4f}")
        else:
            print(f"{key:20}: {value}")

print("\nMatriz de confusión:")
print(cm_df.to_string())

print("\nClassification Report:")
print(report_df.round(4).to_string(index=False))
```

Figura 9: Visualización de Métricas

8. Matriz de confusión

Además de las métricas numéricas, se generó la matriz de confusión, la cual permite observar de manera detallada los aciertos y errores del modelo.

Esta matriz muestra:

- Verdaderos positivos
- Verdaderos negativos
- Falsos positivos
- Falsos negativos

Lo cual facilita el análisis del comportamiento del clasificador.

```python
print("\nMatriz de confusión del mejor modelo:")
print(best_info["confusion_matrix_df"].to_string())
```

Figura 10. Impresión de la matriz de confusión del mejor modelo

9. Comparación de modelos

Finalmente, se generó una tabla comparativa con los resultados obtenidos por cada modelo, lo que permitió identificar cuál presenta mejor desempeño.

Esta comparación es fundamental para seleccionar el modelo más adecuado y justificar su uso en el análisis.

```
================================================================================
6. TABLA COMPARATIVA FINAL
================================================================================
                      Modelo  Accuracy  Precision  Recall  F1-score  Balanced Accuracy    MCC  ROC-AUC  PR-AUC  Log Loss  Train Time (min)  Eval Time
 (min)
SGDClassifier_LogLoss_Mejorado    0.8868     0.8812  0.8942    0.8877             0.8868 0.7738   0.9549  0.9538    0.2742             10.89
  1.07
        ComplementNB_Mejorado    0.8457     0.8430  0.8495    0.8462             0.8457 0.6913   0.9241  0.9232    0.3776             18.11
  9.14
```

Figura 11. Tabla comparativa

Resultados obtenidos

En esta sección se presentan los resultados obtenidos tras el entrenamiento y evaluación de los modelos implementados. Se compararon dos modelos de clasificación: SGDClassifier y Complement Naive Bayes, utilizando el conjunto completo del dataset amazon_polarity.

1. Tabla comparativa de métricas

A continuación, se muestra la tabla comparativa con las métricas obtenidas por cada modelo:


```
================================================================================
6. TABLA COMPARATIVA FINAL
================================================================================
                      Modelo  Accuracy  Precision  Recall  F1-score  Balanced Accuracy    MCC  ROC-AUC  PR-AUC  Log Loss  Train Time (min)  Eval Time
 (min)
SGDClassifier_LogLoss_Mejorado    0.8868     0.8812  0.8942    0.8877             0.8868 0.7738   0.9549  0.9538    0.2742             10.89
  1.07
        ComplementNB_Mejorado    0.8457     0.8430  0.8495    0.8462             0.8457 0.6913   0.9241  0.9232    0.3776             18.11
  9.14
```


Figura 12. Tabla comparativa

2. Análisis de resultados

A partir de los resultados obtenidos, se observa que el modelo **SGDClassifier** presenta un mejor desempeño general en comparación con el modelo Complement Naive Bayes.

El modelo SGDClassifier logra:

- Un **F1-score de aproximadamente 0.8877**, lo cual indica un buen equilibrio entre precisión y recall
- Un **ROC-AUC cercano a 0.95**, lo que demuestra una excelente capacidad de discriminación entre clases
- Un rendimiento consistente en todas las métricas evaluadas

Por otro lado, el modelo Complement Naive Bayes presenta un desempeño inferior, aunque sigue siendo una alternativa rápida y eficiente. Su F1-score es menor, lo que indica una menor capacidad para balancear correctamente los errores entre clases.

En general, los resultados muestran que el uso de un modelo basado en gradiente (SGD) es más adecuado para este tipo de problema y tamaño de dataset.

3. Matriz de confusión

Para analizar con mayor detalle el comportamiento del modelo, se generó la matriz de confusión del mejor modelo (SGDClassifier).

```
Matriz de confusión del mejor modelo:
                   Pred Negativa (0)  Pred Positiva (1)
Real Negativa (0)             175891             24109
Real Positiva (1)              21157            178843
```

Figura 13. Matriz de confusión

El análisis de la matriz de confusión muestra que:

- El modelo clasifica correctamente la mayoría de los ejemplos positivos y negativos
- Existe un número relativamente bajo de errores en ambas clases
- El modelo presenta un comportamiento equilibrado, sin sesgo hacia una clase específica

4. Curvas ROC y Precision-Recall

Se generaron curvas ROC y Precision-Recall para evaluar el desempeño del modelo en distintos umbrales de decisión.

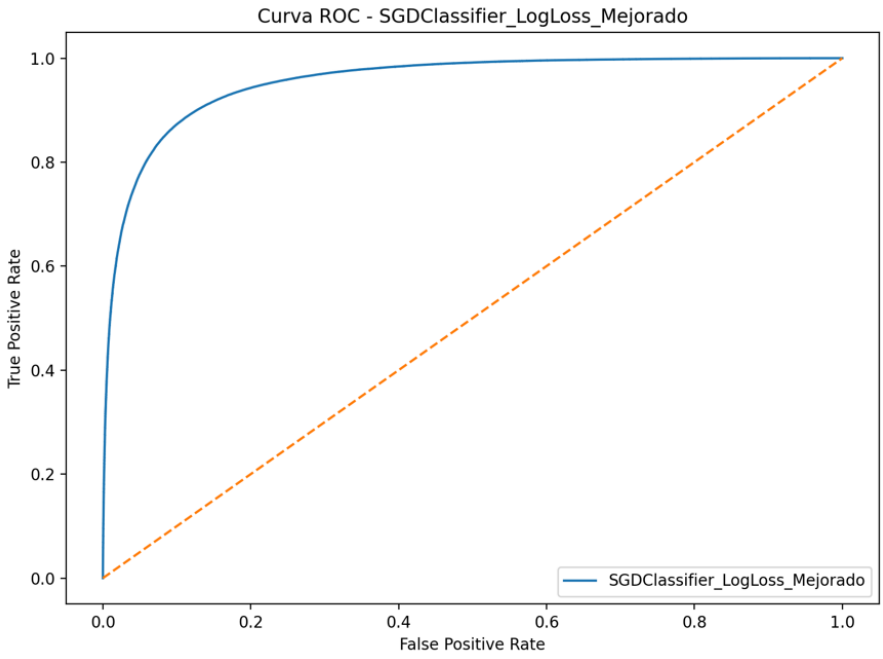

Figura 14. Curva ROC

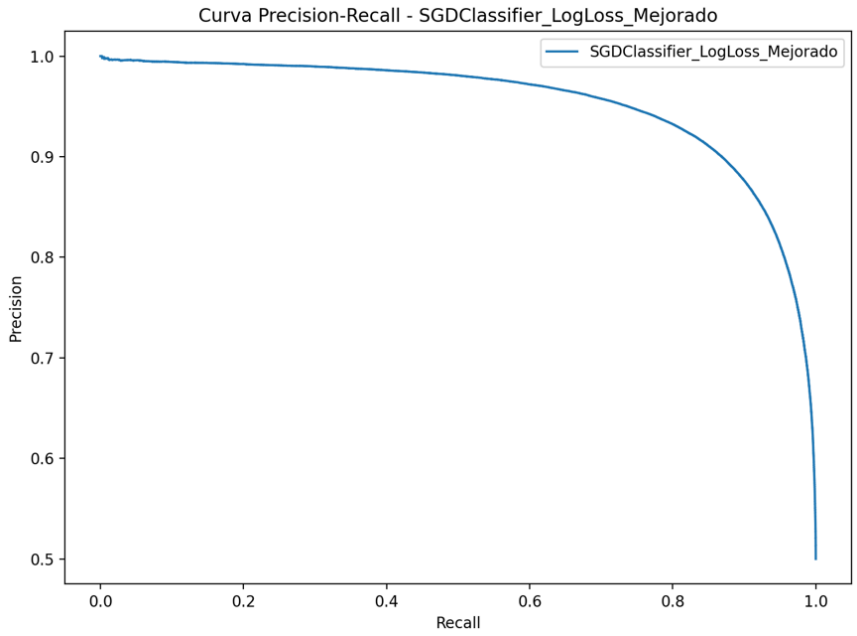

Figura 15. Curva Precision-Recall

5. Análisis de errores

Se realizó un análisis de los ejemplos mal clasificados por el modelo con mejor desempeño. Se identificaron aproximadamente 45,000 errores en el conjunto de prueba, lo cual representa cerca del 11% del total de ejemplos.

El análisis de estos errores permitió identificar patrones como:

- Presencia de lenguaje ambiguo
- Uso de sarcasmo o ironía
- Inconsistencias en el etiquetado del dataset
- Reseñas con palabras positivas en contextos negativos

En algunos casos, se observaron ejemplos donde el modelo asigna una probabilidad muy alta a una clase incorrecta, lo que sugiere que el problema no siempre radica en el modelo, sino en la calidad de los datos.

Este análisis resulta fundamental para comprender las limitaciones del sistema y proponer mejoras futuras.

6. Tiempo de entrenamiento y evaluación

También se midió el tiempo de entrenamiento y evaluación de los modelos. Se observó que:

- El modelo **SGDClassifier** presenta un tiempo de entrenamiento eficiente considerando el tamaño del dataset
- El modelo **Complement Naive Bayes** es más rápido, pero con menor desempeño

Esto confirma que existe un compromiso entre velocidad y precisión en los modelos evaluados.

Conclusiones y recomendaciones

En esta práctica se implementó un sistema de clasificación binaria de texto utilizando el dataset amazon_polarity, con el objetivo de determinar la polaridad de reseñas de productos. Se aplicaron técnicas de procesamiento de lenguaje natural, representación de texto mediante TF-IDF y modelos de aprendizaje automático para resolver el problema planteado.

A partir de los resultados obtenidos, se concluye que el modelo SGDClassifier presenta un mejor desempeño en comparación con el modelo Complement Naive Bayes, logrando un equilibrio adecuado entre precisión y recall, así como valores altos en métricas como F1-score y ROC-AUC. Esto confirma que los modelos lineales basados en gradiente son más adecuados para problemas de clasificación de texto a gran escala.

Además de cumplir con los requerimientos básicos de la práctica, se realizaron diversos experimentos adicionales con el objetivo de mejorar el desempeño del modelo y analizar el impacto de distintas configuraciones. Entre los principales experimentos realizados se encuentran:

- **Uso de bigramas (ngram_range = (1,2))**, lo cual permitió capturar contexto entre palabras y mejorar la interpretación de frases completas.
- **Eliminación de stopwords**, con el fin de reducir ruido en los datos y mejorar la calidad de la representación textual.
- **Ajuste de hiperparámetros del modelo SGDClassifier**, incluyendo el uso de *early stopping*, validación interna y criterios de convergencia, lo que permitió obtener un modelo más estable y evitar sobreajuste.
- **Comparación entre distintos modelos de clasificación**, evaluando tanto un modelo probabilístico (Naive Bayes) como uno basado en optimización (SGD).
- **Evaluación con múltiples métricas avanzadas**, como ROC-AUC, PR-AUC, MCC y Balanced Accuracy, lo cual permitió analizar el desempeño del modelo desde diferentes perspectivas.
- **Análisis de errores**, identificando ejemplos mal clasificados para comprender las limitaciones del modelo y del dataset.

Estos experimentos permitieron mejorar significativamente el desempeño del sistema respecto a una implementación básica, demostrando la importancia de la selección adecuada de características y la optimización de modelos en problemas de NLP.

Asimismo, se observó que el tamaño del dataset influye de manera importante en el desempeño del modelo, ya que el uso de millones de ejemplos permite una mejor generalización, aunque también implica mayores costos computacionales.

Entre las principales limitaciones identificadas se encuentran la presencia de ruido en los datos, así como la dificultad de interpretar correctamente textos con sarcasmo o ambigüedad semántica. Estos factores afectan el desempeño del modelo, independientemente de la técnica utilizada.

Como trabajo futuro, se recomienda explorar modelos más avanzados basados en aprendizaje profundo, como redes neuronales o modelos tipo Transformer (por ejemplo, BERT), los cuales pueden capturar mejor el contexto semántico del lenguaje. También sería posible mejorar el preprocesamiento mediante técnicas como lematización o embeddings más sofisticados.

En conclusión, la práctica permitió comprender de manera integral el proceso de clasificación de texto, desde el preprocesamiento hasta la evaluación, así como la importancia de experimentar con distintas configuraciones para obtener mejores resultados.

Bibliografía

- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Draft version. Disponible en: https://web.stanford.edu/~jurafsky/slp3/
- Aggarwal, C. C. (2018). *Machine Learning for Text*. Springer.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- Scikit-learn Developers. (2024). *Scikit-learn Documentation*. Disponible en: https://scikit-learn.org/
- Hugging Face. (2024). *Datasets Library Documentation*. Disponible en: https://huggingface.co/docs/datasets
- Hugging Face. (2024). *amazon_polarity Dataset*. Disponible en: https://huggingface.co/datasets/amazon_polarity
- Zhang, X., Zhao, J., & LeCun, Y. (2015). *Character-level Convolutional Networks for Text Classification*. Advances in Neural Information Processing Systems (NeurIPS).